

TCP and UDP — Technical Notes

Introduction

TCP (Transmission Control Protocol) and UDP (User Datagram Protocol) are transport-layer protocols used for communication between applications over a network.

Their primary purpose is to provide a mechanism for transferring application data between a **source** and a **destination**.

The fundamental difference is:

- **TCP** provides reliable, ordered, connection-oriented data delivery.
- **UDP** provides connectionless, best-effort data delivery with minimal protocol overhead.

TCP — Transmission Control Protocol

TCP is a **connection-oriented and reliable transport protocol**.

It provides mechanisms to ensure that data is:

- Delivered reliably
- Delivered in the correct order
- Not unnecessarily duplicated
- Retransmitted when necessary
- Controlled according to receiver and network conditions

TCP is therefore suitable for applications where **data correctness is more important than minimizing protocol overhead**.

2.1 Connection-Oriented Communication

TCP is called **connection-oriented** because a logical connection is established between the communicating endpoints before application data is exchanged.

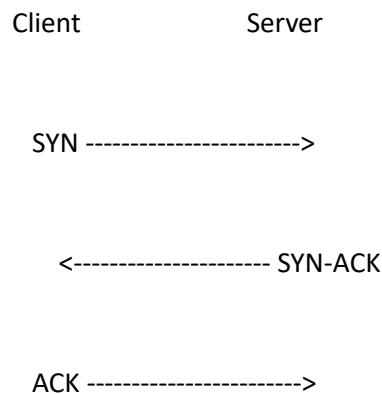
For example:

[illegible]

TCP uses a process called the **three-way handshake** to establish this connection.

3. TCP Three-Way Handshake

The TCP connection establishment process consists of three messages:



SYN

SYN (Synchronize) is used to initiate a TCP connection and synchronize sequence numbers between the two endpoints.

SYN-ACK

SYN-ACK indicates that the server received the connection request and is also acknowledging the client's request.

ACK

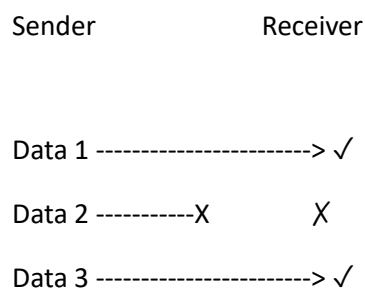
ACK (Acknowledgement) confirms that the client received the server's response.

After this exchange, the TCP connection is established and application data can be transferred.

4. TCP Reliability

One of TCP's primary characteristics is **reliable data delivery**.

Consider:



If Data 2 is lost, TCP can detect the missing data and retransmit it.



This mechanism is called **retransmission**.

Retransmission

Retransmission means sending previously transmitted data again because TCP determines that it was not successfully received.

5. TCP Acknowledgements

TCP uses **ACK (Acknowledgement)** messages to confirm successful receipt of data.

Simplified example:

Sender Receiver

Data ----->

<----- ACK

The ACK tells the sender that the receiver has successfully received the relevant data.

If the sender does not receive the expected acknowledgement within an appropriate period, TCP can take corrective action, such as retransmitting data.

6. TCP Sequence Numbers and Ordering

TCP assigns **sequence numbers** to transmitted data.

Sequence numbers allow TCP to identify the position of data within a stream.

For example, suppose data is transmitted as:

1 → 2 → 3 → 4

It might arrive internally at the receiver in a different order:

1 → 3 → 2 → 4

TCP can use sequence numbers to reconstruct the correct order:

1 → 2 → 3 → 4

Therefore, TCP provides **ordered delivery** to the application.

7. TCP Data Stream

TCP provides a **byte-stream service**.

A byte stream means that the application sees the communication as a continuous sequence of bytes rather than as independent messages.

For example, an application may send:

HELLO

WORLD

TCP treats the data as a continuous stream of bytes:

HELLOWORLD

TCP does not preserve application-level message boundaries by itself.

This is an important distinction when programming with TCP sockets.

8. TCP Flow Control

Flow control prevents a sender from transmitting data faster than the receiver can process or buffer it.

Consider:

Fast Sender -----> Slow Receiver

If the sender continuously transmits data at a very high rate, the receiver's available buffer may become full.

TCP uses mechanisms such as the **receive window** to control the amount of unacknowledged data that can be sent.

Conceptually:

Sender:

"How much data can you currently handle?"

Receiver:

"I can handle this much."

The sender then adjusts its transmission accordingly.

Key point

Flow control protects the receiver.

9. TCP Congestion Control

Congestion control deals with congestion within the network.

Network congestion occurs when too much traffic is being transmitted through network resources, causing increased delay or packet loss.

TCP dynamically adjusts its sending behavior based on network conditions.

Conceptually:

Low congestion



TCP can increase transmission

High congestion



TCP reduces transmission

Key distinction

- **Flow control** → protects the receiver.

- **Congestion control** → responds to network congestion.

10. TCP Characteristics

TCP provides:

Feature	TCP
Connection-oriented	Yes
Reliable delivery	Yes
Ordered delivery	Yes
Acknowledgements	Yes
Retransmission	Yes
Sequence numbers	Yes
Flow control	Yes
Congestion control	Yes
Data model	Byte stream
Protocol overhead	Relatively higher

11. UDP — User Datagram Protocol

UDP is a **connectionless transport protocol** designed to provide a simple mechanism for sending datagrams.

Unlike TCP, UDP does not establish a connection before transmitting application data.

A sender can transmit a UDP datagram directly:

Sender Receiver

Datagram ----->

There is no TCP-style three-way handshake.

12. UDP Datagram

A **datagram** is an independent unit of data transmitted using UDP.

For example:

Datagram 1

Datagram 2

Datagram 3

Each datagram is treated independently.

This differs from TCP's continuous byte-stream model.

13. UDP Reliability

UDP provides **best-effort delivery**.

This means UDP does not guarantee that a datagram will reach the destination.

For example:

Sender Receiver

Datagram 1 -----> ✓

Datagram 2 -----X X

Datagram 3 -----> ✓

UDP itself does not automatically retransmit Datagram 2.

If an application requires retransmission, that functionality must be implemented by the application or by another protocol operating above UDP.

14. UDP Ordering

UDP does not guarantee that datagrams will arrive in the same order in which they were sent.

For example:

Sent:

1 → 2 → 3 → 4

The receiver could observe:

1 → 3 → 4 → 2

UDP does not automatically reorder these datagrams.

If ordering is required, the application must implement the necessary mechanism.

15. UDP Acknowledgements

UDP does not provide TCP-style acknowledgements.

For example:

Sender Receiver

Data ----->

The receiver does not have to send:

<----- ACK

as part of UDP itself.

An application can implement its own acknowledgement mechanism if required.

16. UDP Flow and Congestion Control

UDP itself does not provide TCP-style:

- Flow control
- Congestion control
- Automatic retransmission
- Reliable delivery

This contributes to UDP's simplicity and low protocol overhead.

However, this also means that an application using UDP must be designed carefully if it needs to handle packet loss, congestion, ordering, or reliability.

17. Why Use UDP?

The absence of TCP's reliability mechanisms can be beneficial in applications where **low latency, simplicity, or application-specific control** is important.

Consider a real-time application.

Suppose updates are:

Position 1

Position 2

Position 3

Position 4

If Position 2 is lost, retransmitting the old position may not always be useful because the application has already moved to newer information.

The application may prefer:

Position 1

Position 3

Position 4

rather than waiting for an old update.

This makes UDP suitable for certain real-time communication scenarios.

18. TCP vs UDP

Property	TCP	UDP
Full name	Transmission Control Protocol	User Datagram Protocol
Connection	Connection-oriented	Connectionless
Reliability	Guaranteed by protocol mechanisms	Best-effort
Ordering	Guaranteed	Not guaranteed
Retransmission	Built in	Not built in
ACKs	Built in	Not built in
Flow control	Yes	No
Congestion control	Yes	No
Data representation	Byte stream	Datagrams
Handshake	Three-way handshake	No TCP-style handshake
Overhead	Higher	Lower
Complexity	Higher	Lower

19. TCP and UDP — Conceptual Comparison

TCP

TCP can be summarized as:

"I will provide mechanisms to ensure that your data is delivered reliably and in the correct order."

Its design prioritizes **reliability and correctness**.

Application

↓

TCP

↓

Reliable + Ordered Data

UDP

UDP can be summarized as:

"I will provide a simple mechanism for sending your data, without providing TCP's reliability guarantees."

Its design prioritizes **simplicity and low protocol overhead**.

Application



UDP



Datagrams

Common Use Cases

TCP is commonly used for:

- Web applications
- HTTPS connections
- SSH
- File transfers
- Email protocols
- Database connections

These applications generally require reliable and ordered data.

For example, when downloading a file:

File:

A B C D E F G

Missing data is usually unacceptable.

TCP can retransmit missing data and reconstruct the correct stream.

UDP is commonly used for:

- DNS
- Online gaming
- Real-time communication
- Certain streaming/media applications
- Applications where low latency is important

The exact choice depends on the application's requirements.

Important Terminology

Protocol

A **protocol** is a defined set of rules used for communication between systems.

Endpoint

An **endpoint** represents one side of a communication.

For example:

Application A $\leftrightarrow$ Application B

Endpoint Endpoint

ACK

Acknowledgement — indicates that data has been received.

Retransmission

Sending data again when the original transmission is considered unsuccessful.

Sequence Number

A number used by TCP to identify the position of data within the byte stream.

Connection-Oriented

Communication where a connection is established before normal data transfer.

Connectionless

Communication where data can be sent without establishing a TCP-style connection first.

Datagram

An independent unit of data transmitted using UDP.

Flow Control

A mechanism that prevents a sender from overwhelming the receiver.

Congestion Control

Mechanisms used to control transmission when the network is experiencing congestion.

Overhead

Additional processing, data, and communication required to provide protocol features.

TCP: *Reliability and ordering are provided by the transport protocol.*

UDP: *The application gets a simple datagram service and must handle additional requirements itself if needed.*